

技术分享

进化式编码智能体

让 AI 自主发现并优化算法

AlphaEvolve · 伐谋 FM Agent · OpenEvolve

技术解析 · 上手示例 · 落地思考

面向 AI 技术团队 | 聚焦应用与落地

中交信科集团星宇公司 · 王宇



本次分享议程

01



范式

什么是进化式编码智能体：生成—评分—进化闭环

02



AlphaEvolve

开创者（Google DeepMind）：原理与基础设施落地

03



伐谋 FM Agent

产业化（百度）：架构、基准与产业现场落地

04



OpenEvolve

开源可复现：架构 + 两个最小示例上手

我们的核心业务，本质都是“超大规模优化问题”



交通运输

信号配时 · 路网调度 · 运力与路径优化
(VRP/TSP)



北斗导航

信号处理 · 滤波器设计 · 定位解算与组合导航优化



建筑设计

管线/桥架三维布线 · 结构方案 · 施工排程优化



它们有一个共同点

评估一个方案的好坏很容易，但在天文数字般的可能性里找到**最优方案**极其困难。

——这恰恰是进化式编码智能体**最擅长的事**。



PART 01

什么是进化式编码智能体

让 AI 不止于写代码，而是在解空间中自主搜索、发现更优算法



从 “AI 写代码” 到 “AI 发现算法”



普通 LLM 编码助手

- 一问一答、一次性生成
- 存在幻觉风险，难以保证正确
- 依赖人来判断好坏、反复手动修改
- 本质是 “生成”



进化式编码智能体

- 生成—评分—进化的持续闭环
- 每个方案都被自动验证、客观打分
- 用进化算法在解空间中系统搜索
- 本质是 “搜索 + 发现”

生成—评分—进化：一个自动闭环



一个不可绕过的前提：可自动验证的评估函数

“没有评估函数，就没有进化。” 只适用于“解能被客观、自动打分”的问题。



适用：进展可被系统度量

- 数学与算法问题
- GPU/高性能计算内核优化
- 调度、布线、路由等运筹优化
- 信号处理、滤波器设计



不适用：难以自动判定优劣

- 开放式、主观审美类任务
- 必须靠人工评审才能定好坏
- 需长周期实物实验验证（如药物试验）
- 无明确量化目标的问题

一条清晰的演进脉络



LLM 接管了“变异算子”——把人类难以枚举的代码改写，变成可大规模自动探索的进化操作。



PART 02

AlphaEvolve

Google DeepMind · 2025年5月
Gemini 驱动的进化式编码智能体，从基础设施到前沿数学



Gemini 驱动的通用算法发现与优化



发布

Google DeepMind, 2025 年 5 月, FunSearch 的直接后继



模型集成

Gemini Flash 拓展想法广度 + Gemini Pro 提供深度洞察



能力跃迁

从 “单函数发现” 跃升为 “进化整个代码库、复杂算法”



已落地

已部署于 Google 数据中心、芯片设计与 AI 训练流程

把「自然选择」搬进代码：一个自动进化的闭环

不直接写答案，而像生物进化——不断「变异 → 评估 → 择优」，让代码一代代变好。四个角色在闭环中协作：



🔄 **如此循环往复** 控制器把四者放进异步流水线并行运转，为「吞吐量」（单位算力测试的点子数）而非单次速度优化。

强化学习视角： 状态 = 一段程序，动作 = 修改「可进化区」，奖励 = 评估器给的分。

灵感从哪来：富上下文提示 + 「快慢搭配」的双模型

① 提示采样器：站在巨人的肩膀上改

从程序库挑出「父代程序」(待改进) 与「灵感程序」(多样参照)，拼成一段富上下文提示，再喂给大模型：

〔喂给大模型的提示长这样〕

【题目】 11 维空间最大化 kissing number...

【历史最优】 程序#A · 得分 0.86 · <代码...>

【参考资料】 相关不等式 / 论文...

【指令】 用 SEARCH/REPLACE 改进上面的程序

还能「进化提示词本身」(meta-prompt)，让提示也越用越好。

类比：像发给学生「往届高分作业 + 评分 + 题目」，在已知最优上改，而非从零空想。

② LLM 集成：变异引擎

Gemini Flash 快 · 便宜

→ 广度：单位时间产出海量点子

Gemini Pro 强 · 较慢

→ 深度：偶尔「神来之笔」带来突破

模型无关：换更强的模型，效果更好。

一个比喻

一个爱头脑风暴的「快手」

+ 一个深思熟虑的「专家」

搭配产出：又多又好。

怎样「精准动刀」：只改指定段落，且用「修订」而非重写

① EVOLVE-BLOCK：圈定「可进化区」

用注释标出可被改写的代码段；其余（题目、评估器、胶水代码）全程冻结。

关注点分离 → 能改的只有「策略」，规则锁死，从根上防「作弊」。

② diff 改写：像「修订」，不重写全文

模型只产出「找这段 → 换成这段」的 SEARCH/REPLACE 块。

精准可控，因此能进化「整个代码库」，而不止单个函数。

〔初始程序：只有标记区会变〕

```
# EVOLVE-BLOCK-START
def heuristic(state):
    ...           # ← 只有这段会被改写
# EVOLVE-BLOCK-END

def evaluate(prog): # 评估器·冻结
```

〔一次改动长这样〕

```
<<<<<<< SEARCH
    return adam(lr)
=====
    return adamw(lr, weight_decay=1e-4)
>>>>>>> REPLACE
# 像 Word 修订 / 外科手术，定点修改
```

为什么重要 正因改动是定点 diff，AlphaEvolve 才能在动辄上千行的真实代码库里持续进化——而不只是改一个孤立的小函数。

自动裁判：没有「评分函数」，就没有进化

铁律 你得能写出一个自动、客观打分的 `evaluate()`——这正是 AlphaEvolve「抗幻觉」的根本：一切以分数说话，而非模型的「自我感觉」。

评估器 `evaluate()`

运行候选程序，返回一组可最大化的分数。

```
def evaluate(program):  
    ...  
    return {"valid": 1.0,  
           "score": 0.94}
```

级联评估 · 招聘漏斗

便宜小测试（秒级）

↓ 过关才继续

中等规模测试

↓ 过关才继续

昂贵大规模测试

快速淘汰坏点子，省下算力。

多目标 + 并行

• 同时优化多个指标

（常常反而提升单一目标，并增加多样性）

• 评估跨机器并行

把昂贵评测摊到很多台机器同时跑。

实例 一条进化出的数据中心调度公式（仅几行代码），为 Google 全球机群回收了约 0.7% 算力——小代码，大影响。

记忆与多样性①：MAP-Elites —— 每个「生态位」各留一个冠军

不只保留「最快的那一个」

而是按「特征维度」把解空间切成格子（生态位），每个格子只留当前冠军。

为什么这样做？

保住多样性、避免一头扎进局部最优——今天某个冷门格子里的「次优解」，明天可能跳出真正的全局最优。

一个比喻

像球队「每个位置各留最佳球员」，而不是只留一个最强的人；或养马时「泥地最快 / 短途最快 / 长途最快」各留一匹。

解空间网格：每格只留冠军（★ 及其得分）



→ 代码长度（短 ... 长）

新方案只和「同格子」的冠军比，赢了才替换它。

记忆与多样性②：岛屿模型，与「探索 vs 利用」的精髓

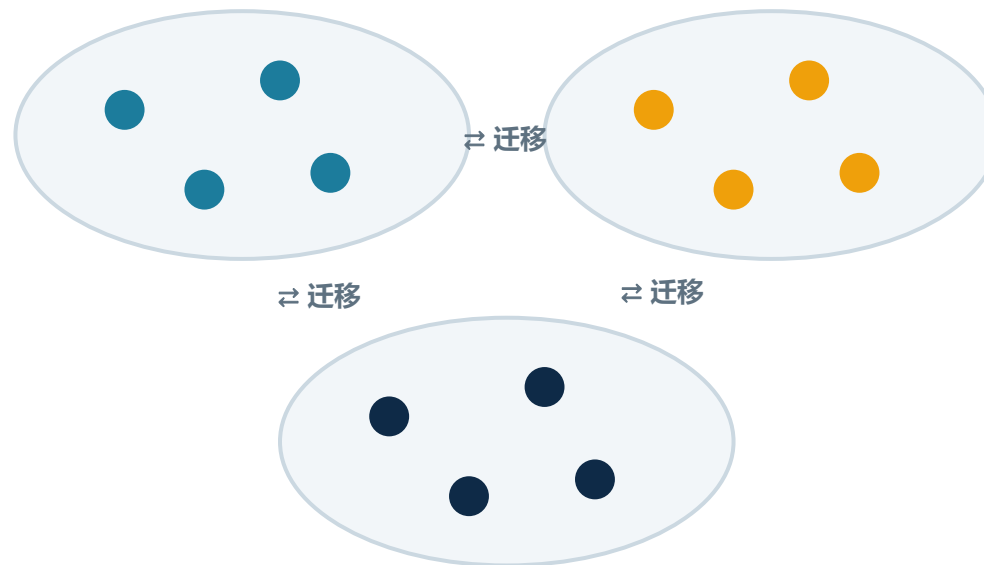
岛屿模型 (Island Model)

多个隔离的种群并行进化，各自长出不同的「算法家族」。

周期性「迁移」：把优秀个体送到别的岛，交流好基因，防止整体早熟、停滞。

一个比喻

像加拉帕戈斯群岛——各岛独立演化出独特物种，偶尔交流。



各岛独立探索，迁移共享突破——既不近亲繁殖，也不全军覆没。

精髓 在「利用已知最优」与「探索新方向」间动态平衡。它真正进化的，往往不是「答案」本身，而是「搜索策略」——自动发现多阶段、自适应的启发式，正是人工最难复制的部分。

数学：刷新尘封多年的纪录

48

次标量乘法

4×4 复矩阵乘法
56 年来首超 Strassen(1969)

20%

改进开放问题

50+ 数学开放问题
75% 追平 SOTA、20% 超越

593

个外球 / 11 维

Kissing Number 问题
刷新长期下界 (原 592)



方法论亮点 它进化的往往不是“答案”本身，而是“搜索启发式算法”——自动发现多阶段、自适应的搜索策略，这正是人工最难复制的部分。

Google 基础设施：真金白银的落地

0.7%

全球算力回收

数据中心调度(Borg)
已生产运行 1 年以上

+23%

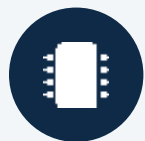
训练内核加速

Gemini 关键 kernel
→ 训练总时长 -1%

+32.5%

GPU 内核提速

FlashAttention 实现
编译器一般不碰的底层



硬件设计 AlphaEvolve 提出的 Verilog 电路改写通过了功能正确性验证，已被集成进下一代 TPU 芯片——“用 AI 帮助设计下一代 AI 芯片”。

从内部走向科学与商业 (2026)



量子物理

Willow 处理器电路误差降 10×



基因组学

PacBio 变异检测错误 -30%



电网优化

可行解率 14% → 88%



金融 / Klarna

大模型训练速度翻倍



物流 / FM Logistic

路由 +10.4%，年省 1.5 万公里



材料 / Schrödinger

MLFF 训练与推理 4× 提速

通过 Google Cloud 早期访问 API 对外开放——已从“实验室成果”变为“可被企业调用的服务”。



PART 03

伐谋 FM Agent

百度智能云 · arXiv 2510.26144

“上兵伐谋”——自我演化决策智能体，全球首个同类商业化产品



面向产业的“自我演化决策智能体”



来源

百度智能云，论文《The FM Agent》（2025.10），沈抖团队



命名

取自《孙子兵法》“上兵伐谋”——以谋略而非蛮力求胜



定位

“产业大脑”，核心场景是为复杂决策找“全局最优解”



节奏

2025 百度世界大会发布，2026 Create 大会升级为 2.0

四大创新：在 AlphaEvolve 骨架上的产业化增强



冷启动 + 专家引导

并行多智能体先撒一张多样的“种子网”，可注入专家经验，缓解早熟



进化阶段

多种群岛屿 + 自适应多样性采样 + 领域特定评估器



Ray 分布式异步

生成与评估分池并行，数千进化进程并发，动态负载均衡



人机交互 / RAG

可视化干预 + 知识库检索，把领域知识喂进进化循环

在权威榜单上登顶

登顶

MLE-Bench

机器学习工程基准
超越微软 R&D-Agent、OpenAI AIDE

+5.2%

ALE-Bench

组合优化基准
Lite 得分 1976.3

2–20×

KernelBench

GPU 内核优化
相对 torch.compile 加速



国产首次 MLE-Bench 此前鲜见国内产品；伐谋 2.0 在 15 道最难题目中拿下 9 项第一，并在多个经典数学问题（圆装填等）上确立新 SOTA。

已“入职”产业现场——且与我们业务高度重合



交通信控

鄂尔多斯伊金霍洛旗：上千红绿灯配时求“全局最优”



金融风控

中信百信银行风控系统



海上风电（工程）

三维电缆布线“迷宫”：比人工更短路径，交付省数倍时间



生产排程

404 任务→82 台注塑机，生产率 +6.31%



物流规划

自动化码头 A-TOS 系统绝对指标 +10.21%



工艺优化

水稻育种检测效率提升 90%+

高亮项 = 与中交星宇业务直接对应：交通信号、三维布线，伐谋已有现成落地。

怎么用：业务专家用自然语言， 步拿到最优方案

自然语言驱动复杂优化，业务专家无需懂算法或编程；适用于生产排程、物流调度、需求预测、工艺优化等多目标、多约束场景。



无需编程，零门槛 官方明确「不需要懂算法或编程」——产业决策智能体把建模与评估自动化，封装在自然语言之下。下两页以「离散制造·两周排产」为例，拆解提示词级、数据/代码级的真实输入与结果解读。

案例：离散制造两周排产——你只需填一段结构化提示词

场景 ≈ 50 台设备 · 双班制 400+ 积压订单 · 含 VIP 优先级 2 周 排程窗口 · 精确到小时

① 自然语言提示词（用户直接输入）

我的角色：离散制造工厂 · 生产计划员

问题描述：未来两周、约 50 台设备、400+ 积压订单、双班制排产

当前数据（已上传）：订单表（交期 / 数量 / VIP 优先级）、设备表（产能 / 换型时间 / 维护窗口）

优化目标：在满足 VIP 与交期前提下，提高设备利用率、降低换型成本

不可突破的限制：维护停机窗口不可占用；A 产线不可跨班生产

期望输出：精确到小时的排程甘特图 + 关键指标对比表

提示：描述越完整，澄清轮次越少；字段照填即可，无需固定格式。

② 自动澄清示例（10–30 秒）

Q 「成本、效率、交期，哪个优先？」

A 交期 > 利用率 > 换型成本

Q 「维护窗口是否为硬约束？」

A 是，停机窗口不可占用

Q 「VIP 如何界定？」

A 订单表 priority=VIP 列

↳ 关键词作答即可，系统自动补全上下文并进入推演。

「代码级别」已被自动化：你只给数据，再读懂报告、用追问迭代

① 数据级别：上传结构化文件（零代码）

`orders.csv` 订单号 / 交期 / 数量 / 优先级(VIP)

`machines.csv` 机台 / 产能 / 换型时间 / 维护窗口

`routing.csv` 工序 / 标准工时（可选）

点击 / 拖拽 / 粘贴 CSV；单任务 ≤ 20 文件、 $\leq 500MB$ 。

③ 读报告：结构化结果 + 可视化

- 核心结论与最优排产方案
- 关键指标：利用率 $\uparrow$ / 换型 $\downarrow$ / 准时交付率
- 约束满足情况 + 风险与异常提示
- 关键影响因素分析 + 可执行建议
- 可视化：小时级排产甘特图

② 代码级别：已被 Agent 自动化

在「管理平台」中，才需算法工程师手写：

`solver.py` # 待改进算法（仅 .py）

EVOLVE-BLOCK-START/END ← 圈定可进化片段

`evaluator.py` → validity / score

产业决策智能体把这层「建模 + 评估」全自动化——业务专家无需编写或修改任何代码。

④ 追问即迭代（提示词 / 数据级的「修改」）

- 「增加一台设备会怎样？」
- 「为什么推荐这组参数？」
- 「哪些变量影响最大？」
- ↳ 无需重传文件，基于上下文继续推演，报告与甘特图同步刷新。



PART 04

OpenEvolve

Asankhaya Sharma / ASI Labs · 2025.5

唯一完全开源、可复现、模型无关的实现——人人可上手



五组件闭环 + 工程化亮点



沿用 AlphaEvolve 骨架

- Prompt Sampler / LLM Ensemble / Evaluator
- Program Database / Controller (异步)
- MAP-Elites + 岛屿模型 (惰性迁移、环形拓扑)



级联评估

分阶段过滤，先便宜后昂贵



产物旁路通道

把 stderr/错误反馈带入下一代提示



双重选择

父代取高分、灵感取多样，分离探索与利用



可复现 + 多语言

seed=42; 支持 Python/Rust/R/Metal 等

看两个最小例子：从“不会”到“学会”

examples/ 里有几十个例子 挑两个最好懂的——一个连续优化、一个离散算法，把“进化”讲透。



例 1 • function_minimization

- 连续优化：找布满“坑”函数的全局最低点
- 起点：纯随机撒点取最低（很朴素）
- 进化出：模拟退火 + 自适应降温
- 看点：算法范式的自动跃迁



例 2 • online_judge / Kattis

- 离散算法：把字符串变“字母表有序”
- 起点：直接 return 0（根本不会做）
- 进化出：可证明正确、通过全部判题
- 看点：从“0分”到“满分”的发现

把“坑坑洼洼”的函数推到全局最低点



任务

最小化 $f = \sin x \cdot \cos y + \sin(xy) + (x^2 + y^2)/20$ ——遍布局部极小值



可进化区

EVOLVE-BLOCK 圈定可改区；评分函数 `evaluate_function` 固定不动



起点程序

纯随机搜索：不停撒点、保留最低值；盲目，易被局部坑卡住



评估器

多次运行，按“最终最小值 + 是否稳定收敛”客观打分

没人教“模拟退火”，它自己进化了出来

100×

收敛速度提升

对比纯随机搜索
同预算、同目标函数

0 行

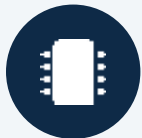
人工算法逻辑

只给了起点与评分
算法由进化自动写出

1 个

固定的评估函数

它定义了进化方向
“没有评估就没有进化”



进化出了什么 从随机搜索 → 模拟退火（含自适应降温）：以一定概率接受更差解，才能跳出局部坑。这正是我们做信号配时 / 路径寻优可借鉴的范式。

让字符串“字母表有序”，最少要补几个字母？



任务

给定小写串 s (≤ 50)，求最少插入几个字母，使其包含子序列 $a \rightarrow b \rightarrow \dots \rightarrow z$



举例

aiemckgobjfndlhp $\rightarrow$ 需补 20 个；已有序的部分越长，要补的越少



起点程序

一个“假”程序：直接 return 0——完全不会做，只是占位



评估器

提交到 Kattis 在线判题 / 用测试用例比对，“过不过”就是最硬的评分

从 return 0 进化到一个可证明正确的解法



进化发现的解法

- 核心洞察：答案 = $26 - L$
- L = 最长“字母递增”子序列长度
- 即经典 LIS（最长上升子序列）
- 自动通过 Kattis 全部用例 ✓



对我们的启示

- 起点完全不会做，只靠“对 / 错”反馈
- 也能进化出可证明正确的算法
- 离散、逻辑型问题同样适用
- **凡“对错可判定”的算法题，皆可从零起步**

建议的四步走（务实、低风险）



一句话：先把“评估器”做好，AI 才能进化——评估函数的质量，直接决定结果的天花板。

清醒地看待它的局限



评估函数依赖

无法自动打分的问题用不了——这是硬边界



算力与调用成本

需控制迭代与模型规模；可用本地模型降本



高度依赖提示工程

需注入领域知识、反复调优系统提示



结果需独立验证

产业宣传数字需打折；必须用我们自己的基准复核



定位它：它不替代工程师，而是放大工程师——把专家从耗时的试错中解放出来，专注于问题定义与策略判断。

总结与行动建议



一个范式

LLM + 进化搜索 + 自动评估 = 自主算法发现，寻优而非问答。AlphaEvolve 堪称 OpenAI 能力五阶的第 4 阶「创新者（创新辅助者）」——一台自动化创新引擎。



已被验证

从 Google 基础设施，到伯克利系统研究，到百度产业现场落地



对我们

三大业务高度契合；建议从一个可验证的“配时/布线”痛点，用 OpenEvolve 低成本起步

讨论 & 答疑 | 谢谢